Final Project for Introduction to Programming

Université Jean Monnet – Saint-Étienne 2026/2027

A General project instructions

A.1 Groups

This project is meant to be completed by pairs of people. You should **form groups of two people**. Should there be a person ending up alone, the forming of a group of three will be allowed.

As soon as your 2-person group of is formed, please do email me at: nicolas.gry@inria.fr with the emails from both peers.

A.2 Project Submission

The due date is: **November 13th of 2026, 23:59:59**. Two things are expected for the project submission:

- **Project archive (one per group):** You will submit an archive containing all the files needed in order for the project to compile, as well as any extra files you may want to add in order for your project to work (e.g extra image files).
- **Small report (one per group):** A report of **three pages maximum** per group is expected. The report should reflect each person's involvement in the project, as well as their struggles and success. The report should contain the following information:
  - What was the original idea (if it differed from the end result) for your project
  - What was tried and failed
  - If applicable, whether or not Gen AI was used, and where/when it was used
  - Explain briefly what the end result program does and how it does it

The submission should be sent by email at: nicolas.gry@inria.fr.

A.3 Expectations

The project is split in three different levels of difficulty. Everyone should complete at least level 1. Those who are more comfortable should try to complete level 2. Level 3 is for anyone who wishes to experiment further.

The current situation (university blockage, at home project, etc) will obviously be taken into account, so don't worry too much about the grading. As long as you try your best on this project, you should pass the class.

Since the project will be completed without diligent surveillance, I have to talk about two things: **generative AI** and **plagiarism**.

On Generative AI first: I won't forbid it (again, the tools to check its use don't exist). Like for the assignments, I will ask you to be honest about its usage, i.e tell if you use it and where. I won't discriminate its usage, i.e you will not get a lesser grade because you did or didn't use it.

On plagiarism: I made this project so it allows for at least a little bit of creativity, so I expect each group to produce their very own program. I will be much more regarding on plagiarism than generative AI. Plagiarism is the one thing that is not allowed. That does not mean you cannot help each other. Just don't copy-paste some other group's code and pretend it is your own. If blatant plagiarism is detected, all involved groups will be sanctioned.

B Project subject

The goal of this project is to create programs that given one or several input images, produce a resulting image. The idea is to create a program that assembles images together, crops them, rotates them, changes their colors, etc. To do so, functions have been implemented and are provided for you to use. They are described at length in the section “Image manipulation functions”. Please, if anything is unclear about the following, do email me at nicolas.gry@inria.fr.

B.1 Getting started

B.1.1 Settings things up

Download the archive . Decompress the archive and open the folder “`sources`” in Visual Studio Code.

As a reminder, a guide is available for Windows users to setup things on their machine at: `https://grybouilli.github.io/courses/intro_to_programming/windows_build_tools_installation_steps.pdf`.

MacOS users should execute the following commands in a terminal to have things installed (each line is a separate command):

```
xcode-select --install
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install --cask visual-studio-code
brew install cmake
```

Once the project is open in Visual Studio Code, open a terminal in VSCode and run the commands to compile the project (each line is a separate command):

Listing 2:

```
cmake -B build
cmake --build build
```

On MacOS, the executables will be located in the `build` folder, like so:

```
build/
├── example_my_image_processor_lvl1
├── example_my_image_processor_lvl2
├── example_my_image_processor_lvl3
├── my_image_processor_lvl1
├── my_image_processor_lvl2
└── my_image_processor_lvl3
```

On Windows, the executables will be located in the folder `build\Debug`, like so:

```
build/
└── Debug/
    ├── example_my_image_processor_lvl1.exe
    ├── example_my_image_processor_lvl2.exe
    ├── example_my_image_processor_lvl3.exe
    ├── my_image_processor_lvl1.exe
    ├── my_image_processor_lvl2.exe
    └── my_image_processor_lvl3.exe
```

B.1.2 The examples

For each level of the project (which are detailed later on), some example program is given. The folder `examples` contains the source code for each level’s example. The archive also contains two images to use for testing purpose in the folder `media`.

B.2 Goals

Here, the actual objectives for this project are decribed. This project is split in three levels. It is advised to try to complete them in the order. Again, a well executed level 1 is already a great achievement.

The projects aims for you to produce a program that processes images. You can use different functions already implemented to do so. The available operations are:

- Swapping the color channels of an image (with `swap_channels`)
- Converting an rgb image to a black and white image (with `convert_rgb_to_bw`)
- Creating an image filled with one color (with `create_blank_image`)
- Concatenating two images, horizontally or vertically (with `concatenate`)
- Cropping an image (with `crop_image`)
- Overlapping an image with another one (with `superposition`)
- Rotating an image by 90 degrees clockwise or counter clockwise (with `rotate90`)

The functions are described in more detail in section C where some example usage are provided for each.

In the following section, the commands provided are for MacOS. Adapt the commands for Windows by adding a "`.exe`" suffix to the executables, and changing the path "`build/`" to "`build/Debug`".

B.2.1 Level 1

The goal is to produce a program that has a single purpose. It should process an input image (or more), and produce an output image. The program, once compiled, can be executed for example like so:

```
./build/my_image_processor_lvl1 media/boat.png media/output.png
```

The example program for this level is a program that takes two input images and concatenates them horizontally. It saves the resulting image in the provided output path. The code for that example is in `examples/example_level1.cpp`. You can try using the program with:

```
./build/example_my_image_processor_lvl1 media/boat.png media/sunflower.png media/output.png
```

Use at least three functions amongst the ones described from C.6 to C.12 to create a program that processes input images and saves the result in the provided output path (you'll note the example uses only `concatenate` once). Complete this level in the file `src/level1.cpp`. Use the cmake command in Listing 2 to compile the project.

B.2.2 Level 2

The goal is to produce a program that has several purposes. It should take an input command, process an input image (or more) according to the given command, and produce an output image. The program, once compiled, can be executed for example like so:

```
./build/my_image_processor_lvl1 mycommand media/boat.png media/output.png
```

The example program for this level is a program that takes one image and converts it to black and white. The command to give is "`tobw`". It saves the resulting image in the provided output path. The code for that example is in `examples/example_level2.cpp`. You can try using the program with:

```
./build/example_my_image_processor_lvl2 tobw media/boat.png media/output.png
```

Implement at least two different commands to create a program that can process images differently upon being given different commands (note the example implements only one command). It is allowed to reuse the code from level 1 as one of the commands. Complete this level in the file `src/level2.cpp`. Use the cmake command in Listing 2 to compile the project.

B.2.3 Level 3

The goal is to enrich the set of possible operations. You will build up on the code written in level 2 to produce a program that has several purposes. As in level 2, the produced program should take an input command, process an input image (or more) according to the given command, and produce an output image. The program, once compiled, can be executed for example like so:

```
./build/my_image_processor_lvl1 mycommand media/boat.png media/output.png
```

To implement new operations, you should note that:

- `Image` is an alias for the type `std::vector<std::vector<std::vector<int>>>`
- An image can be indexed with `image[color_channel][y_coordinate][x_coordinate]`

In the example program for this level, a function that sets a pixel to a certain color in an image has been implemented. Then, a command uses this function to draw a rectangle in an input image and saves the result. The code for that example is in `examples/example_level3.cpp`. You can try using the program with:

```
./build/example_my_image_processor_lvl3 tobw media/boat.png media/output1.png
./build/example_my_image_processor_lvl3 set_pixels media/boat.png media/output2.png 10 10 120
    200 125 30 200
```

Implement at least one operation that modifies or create an image and use it to implement a new command for your program. The code from level 2 should be reused as a foundation here. Complete this level in the file `src/level3.cpp`. Use the cmake command in Listing 2 to compile the project.

C Image manipulation functions

The following functions let you load, create, inspect, and modify images. An image is made of pixels. Each pixel has three color components: red, green, and blue (RGB). These functions are described below in terms of what they are useful for and how you can use them.

C.1 `Channel channel_from_str(const std::string channel);`

Purpose: Turns the name of a color channel into the corresponding `Channel` value. This is useful when the channel name is provided as text.

Example:

```
Channel channel = channel_from_str("red");
```

The variable `channel` now represents the red channel. The accepted spelling of channel names depends on the function's implementation.

C.2 `int get_image_width(const Image& image);`

Purpose: Gets the width of an image, measured in pixels. If the image is empty, the function returns `-1`.

Example:

```
Image image = read_image("input.png");
int width = get_image_width(image);
```

After these lines, `width` contains the image's width in pixels.

C.3 `int get_image_height(const Image& image);`

Purpose: Gets the height of an image, measured in pixels. If the image is empty, the function returns `-1`.

Example:

```
Image image = read_image("input.png");
int height = get_image_height(image);
```

After these lines, `height` contains the image's height in pixels.

C.4 `Image read_image(const std::string filename);`

Purpose: Loads an RGB image from a file and returns that image.

Example:

```
Image image = read_image("input.png");
```

The variable `image` can now be passed to other image functions.

C.5 `void write_image(const std::string filename, const Image& image);`

Purpose: Saves an image to a PNG file.

Example:

```
Image image = read_image("input.png");
write_image("copy.png", image);
```

This saves the image to `copy.png`; the image in memory is not changed.

C.6 `Image swap_channels(const Image& image, Channel c1, Channel c2);`

Purpose: Exchanges two color channels throughout an image. It returns a new image and leaves the original unchanged.

Example:

```
Image image = read_image("input.png");
Image swapped = swap_channels(image, Channel::Red, Channel::Blue);
```

In `swapped`, the red and blue components are exchanged for every pixel. The original `image` remains unchanged.

C.7 `void convert_rgb_to_bw(Image& rgb_image);`

Purpose: Converts an RGB image to grayscale by replacing each pixel's three color components with their average. The image is modified directly.

Example:

```
Image image = read_image("input.png");
convert_rgb_to_bw(image);
write_image("grayscale.png", image);
```

The saved image is grayscale. The function does not create a separate result image; it changes `image` itself.

C.8 `Image create_blank_image(const int width, const int height, const int red = 0, const int green = 0, const int blue = 0);`

Purpose: Creates a new image with a chosen width, height, and solid color. The color components range from 0 to 255; black is the default color.

Example:

```
Image black = create_blank_image(320, 200);
Image red   = create_blank_image(320, 200, 255, 0, 0);
```

The first image is black and the second is red. Both images are 320 pixels wide and 200 pixels high.

C.9 `Image concatenate(const Image& image1, const Image& image2, ConcatMode mode);`

Purpose: Combines two images into a new image. `Horizontal` places them side by side; `Vertical` places one above the other. If the images have different sizes along the dimension that must line up, the missing area is filled with black. The input images are not changed.

Example:

```
Image left  = read_image("left.png");
Image right = read_image("right.png");
Image side_by_side = concatenate(left, right, ConcatMode::Horizontal);
write_image("combined.png", side_by_side);
```

The result contains `left` and `right` side by side. To place one above the other, use `ConcatMode::Vertical` instead.

C.10 `void crop_image(Image& image_to_crop, int top_left_x, int top_left_y, int width, int height);`

Purpose: Keeps only a rectangular part of an image. The rectangle is specified by its top-left position, width, and height. Parts outside the image are clipped; if the rectangle does not overlap the image, the result is empty. The original image is modified.

Example:

```
Image image = read_image("input.png");
crop_image(image, 10, 20, 100, 80);
write_image("crop.png", image);
```

This keeps the rectangle beginning 10 pixels from the left and 20 pixels from the top, with a requested width of 100 pixels and height of 80 pixels.

C.11 `Image superposition(const Image& background, const Image& top_image, int top_image_x, int top_image_y);`

Purpose: Places one image on top of another. The background keeps its original size, and the top image is positioned using its top-left corner. Parts of the top image outside the background are clipped. Both inputs remain unchanged; the combined image is returned separately.

Example:

```
Image background = read_image("background.png");
Image logo       = read_image("logo.png");
Image result = superposition(background, logo, 20, 30);
write_image("with_logo.png", result);
```

The logo is placed with its top-left corner 20 pixels from the background's left edge and 30 pixels from its top edge.

C.12 `void rotate90(Image& image, bool clockwise);`

Purpose: Rotates an image by 90 degrees, clockwise or counter-clockwise. The image is modified directly.

Example:

```
Image image = read_image("input.png");
rotate90(image, true);  // Rotate clockwise
write_image("rotated.png", image);
```

Pass `false` instead of `true` to rotate counter-clockwise.